

31/09/2026

2. Print Left View of a Binary Tree

```
import java.util.*;

class LeftView {

    static void leftView(Node root) {

        if (root==null)

            return;

        Queue<Node> q = new LinkedList<>();

        q.offer(root);

        while (!q.isEmpty()) {

            int size = q.size();

            for (int i = 0; i < size; i++) {

                Node curr = q.poll();

                if (i == 0)

                    System.out.print(curr.data + " ");

                if (curr.left != null)

                    q.offer(curr.left);

                if (curr.right != null)

                    q.offer(curr.right);

            }

        }

    }

}
```

3. Print Right View of a Binary Tree

```
import java.util.*;

class RightView {

    static void rightView(Node root) {

        if (root == null)

            return;

        Queue<Node> q = new LinkedList<>();

        q.offer(root);
```

```

while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        Node curr = q.poll();
        if (i == size - 1)
            System.out.print(curr.data + " ");
        if (curr.left != null)
            q.offer(curr.left);
        if (curr.right != null)
            q.offer(curr.right);
    }
}
}

```

4. Lowest Common Ancestor (LCA) in Binary Tree

```

class LCA {
    static Node findLCA(Node root, int n1, int n2) {
        if (root == null)
            return null;
        if (root.data == n1 || root.data == n2)
            return root;
        Node left = findLCA(root.left, n1, n2);
        Node right = findLCA(root.right, n1, n2);
        if (left != null && right != null)
            return root;
        return (left != null) ? left : right;
    }
}

```